1.栈(stack)

栈是限定仅在表尾进行插入和删除操作的线性表

栈顶(top)：允许插入和删除的一端，一般会有一个top指针指向当前的栈顶元素

栈底(bottom)：不允许插入和删除的一端(表头、固定的)

空栈：栈中没有数据元素

进栈或入栈：元素的插入

出栈或退栈：元素的删除

栈又称为后进先出(Last In First Out)的线性表,简称为LIFO结构

练习：

1.若元素的进栈序列为1234，能否得到3124的出栈序列

2.用s表示进栈，x表示出栈，若元素的进制序列为1234，为了得到1342的出栈序列，给出相应的s和x的操作串

1.1栈的基本操作

栈的抽象数据类型的定义

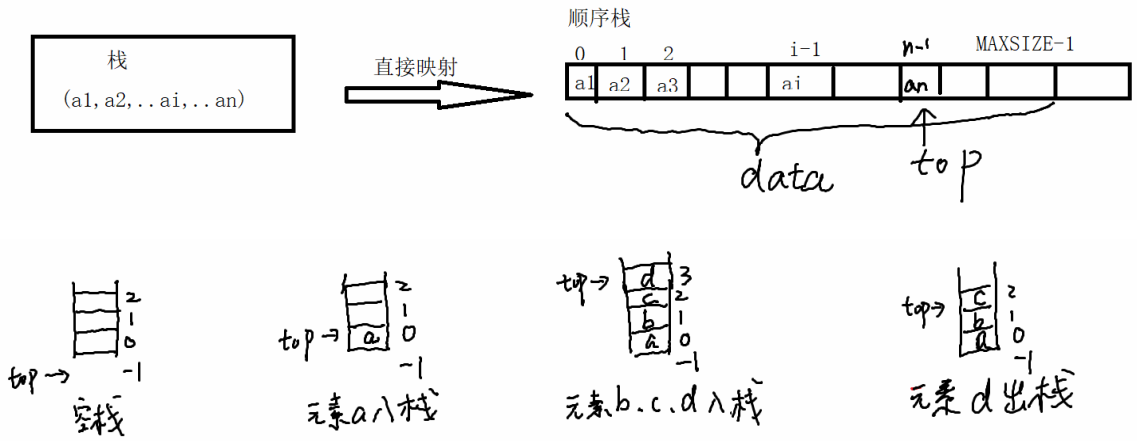
```
数据对象
    D = {ai|1<=i<=n,n>=0,ai为selelmtype类型} //selelmtype为自定义类型
数据关系
    R = {<ai,ai+1>|ai、ai+1 属于D,i=1,...n-1}
基本运算
    初始化操作，建立一个空栈
    判断栈是否为空
    清空一个栈
    销毁一个栈
    获取栈顶元素
    入栈
    出栈，返回出栈元素的值
    获取栈中元素的个数
```

1.2栈的实现

a.顺序栈：sequence stack(seqstack)

用一组连续的存储单元来一次存储栈中的每一个元素(数组)

```c
#define MAXSIZE 50
typedef int selemtype;//数据元素中数据的类型
typedef struct seqstack
{
    selemtype data[MAXSIZE];//用来存储栈中每一个数据
    int top;//栈顶，保存栈顶元素的下标
        //top == -1，表示栈中没有数据元素，空栈
}seqstack;
```



入栈：
top++
在把元素放在top指向的下标的位置

出栈
先获取出栈元素的值
top--

```c
#include <stdio.h>
#include <stdlib.h>
#include "seqstack.h"

/*
    初始化操作，建立一个空栈s
    创建一个空栈，由s指向它
    实际上就是分配一个顺序栈的空间，并将栈顶指针设置为-1
*/
seqstack *init_stack()
{
    seqstack *s = (seqstack *)malloc(sizeof(seqstack));//分配一个顺序栈的空间，首地址存放在s
    s->top = -1;//栈顶指针置为-1,表明没有存放数据
    return s;//返回s中存储的地址
}

/*
    若栈为空,返回true,否则返回false
*/
bool stack_is_empty(seqstack *s)
{
    //栈不存在 || 一个数据元素都没有
    if(s == NULL || s->top == -1)
        return true;
    return false;
```

```c
}

/*
    将栈清空
    栈中没有数据就相当于清空
*/
void clean_stack(seqstack *s)
{
    //栈如果不存在 直接返回
    if(s == NULL)
        return ;

    //栈存在，把栈变回初始化的样子即可
    s->top = -1;//没有数据
}
/*
    若栈存在，则销毁它
    释放栈s占用的存储空间
*/
seqstack *destroy_stack(seqstack *s)
{
    //把申请的空间free即可
    if(s!=NULL)
    {
        free(s);
        s = NULL;
    }

    return s;
}

/*
    入栈
    返回值
        true 可以入栈
        false 不可以入栈
*/
bool push(seqstack *s,selemtype e)
{
    //栈不存在 或 栈满 不能入栈
    if(s==NULL || s->top ==MAXSIZE-1)
        return false;

    //入栈操作
    s->top++;//s->top = s->top+1;  //先让top+1
    s->data[s->top] = e;//再把数据放到top指向的位置
    return true;
}
/*
    出栈
    返回值
        true 可以出栈
        false 不可以出栈

    调用
        selemtype e;
        pop(s,&e)
*/
```

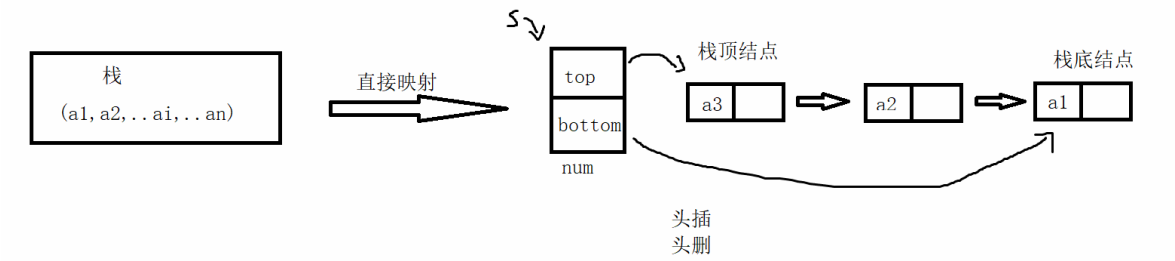
```c
bool pop(seqstack *s,selemtype *d)//d = &e  ==> *d = *&e = e
{
    //调用判断栈是否为空的函数,若为true,不能出栈
    if(stack_is_empty(s) == true)
        return false;
    //出栈
    *d = s->data[s->top];
    s->top--;
    return true;
}

/*
    获取栈顶元素的值
    返回值
        true 可以获取栈顶
        false 不可以获取栈顶

    调用
        selemtype e;
        get_stack_top(s,&e)
*/
int get_stack_top(seqstack *s,selemtype *d)
{
    //调用判断栈是否为空的函数,若为true,不能出栈
    if(stack_is_empty(s) == true)
        return false;
    //出栈
    *d = s->data[s->top];
    return true;
}

/*
    获取栈中元素的个数
*/
int get_stack_length(seqstack *s)
{
    if(s == NULL)
        return 0;
    return s->top+1;
}
```

```c
#include <stdio.h>
#include "seqstack.h"

int main()
{
    //初始化一个栈
    seqstack *s = init_stack();

    selemtype d;
    while(1)
    {
        //输入数据 输入0结束
        scanf("%d",&d);
        if(d == 0)
            break;
```

```
        //入栈
        push(s,d);
    }

    //获取栈顶元素
    if(get_stack_top(s,&d) == true)
        printf("栈顶元素的值为%d\n",d);

    //获取栈中元素的个数
    printf("栈中元素个数为%d\n",get_stack_length(s));

    //循环出栈
    printf("出栈:");
    while(!stack_is_empty(s))//如果栈不为空就循环出栈
    {
        //出栈
        if(pop(s,&d) == false)//出栈失败 提前跳出
            break;

        //打印出栈的值
        printf("%d ",d);
    }

    //销毁栈
    s = destroy_stack(s);
}
```

b.链式栈

可以用一组连续或不连续的存储空间存储每一个元素



需要使用栈顶指针top和栈底指针bottom两个指针

```
//数据类型的定义
typedef int selemtype;
//栈中的数据结点类型
typedef struct node
{
    selemtype data;//数据
    struct node *next;//存放下一个结点的地址
}node;
//链式栈类型
typedef struct linkqueue
{
    node *top;//栈顶指针
    node *bottom;//栈底指针
    int num;//数据结点的个数
}linkqueue;
```

```c
#include <stdio.h>
#include <stdlib.h>
#include "linkstack.h"

/*
    初始化操作，建立一个空栈s
    创建一个空栈，由s指向它
*/
linkstack *init_stack()
{
    //为s创建空间
    linkstack *s = malloc(sizeof(linkstack));
    //为你的s里面的东西进行赋值
    s->top = s->bottom = NULL;
    s->num = 0;
    //返回s
    return s;
}

/*
    若栈为空,返回true,否则返回false
*/
bool stack_is_empty(linkstack *s)
{
    //栈不存在 || 栈顶为NULL || 数据结点的个数为0
    if(s == NULL || s->top == NULL || s->num == 0)
        return true;

    return false;
}

/*
    将栈清空
    栈中没有数据就相当于清空
*/
void clean_stack(linkstack *s)
{
    //判断栈是否为空，如果为true ,直接返回
    if(stack_is_empty(s) == true)
        return ;

    //出栈(头删)
    while(s->top)
    {
        //1.为你要删除的结点起个名字
        node *p = s->top;
        //2.更新s->top
        s->top = p->next;
            //发现s->top == NULL,还需要s->bottom=NULL
        if(s->top == NULL)
            s->bottom = NULL;
        //3.孤立要删除的结点
        p->next = NULL;
        //4.删除
        free(p);
        p = NULL;
        //5.个数--
```

```
        s->num--;
    }

}
/*
    若栈存在，则销毁它
    释放栈s占用的存储空间
*/
linkstack *destroy_stack(linkstack *s)
{
    if(s!=NULL)
    {
        //清空栈
        clean_stack(s);
        free(s);
        s = NULL;
    }
    return s;
}

/*
    入栈
    返回值
        true 可以入栈
        false 不可以入栈
*/
bool push(linkstack *s,selemtype e)
{
    //如果栈不存在 返回false
    if(s == NULL)
        return false;

    //栈存在  入栈(头插)

    //创建结点pnew
    node *pnew = malloc(sizeof(node));
    pnew->data = e;
    pnew->next = NULL;

    //头插
        //如果栈中没有数据结点，新的节点会成为你的top和bottom
    if(s->top == NULL)
    {
        s->top = s->bottom = pnew;
    }
        //如果有数据结点，做头插
    else
    {
        pnew->next = s->top;
        s->top = pnew;//更新top
    }

    //个数+1
    s->num++;
    return true;
}
/*
    出栈
```

```
    返回值
        true 可以出栈
        false 不可以出栈

    调用
        selemtype e;
        pop(s,&e)
*/
bool pop(linkstack *s,selemtype *d)
{
    //判断栈是否为空，如果为true ,直接返回
    if(stack_is_empty(s) == true)
        return false;

    //出栈(头删)

    //获取数据
    *d = s->top->data;

    //1.为你要删除的结点起个名字
    node *p = s->top;
    //2.更新s->top
    s->top = s->top->next;
        //发现s->top == NULL,还需要s->bottom=NULL
    if(s->top == NULL)
        s->bottom = NULL;
    //3.孤立要删除的结点
    p->next = NULL;
    //4.删除
    free(p);
    p = NULL;
    //5.个数--
    s->num--;
    return true;
}

/*
    获取栈顶元素的值
    返回值
        true 可以获取栈顶
        false 不可以获取栈顶

    调用
        selemtype e;
        get_stack_top(s,&e)
*/
bool get_stack_top(linkstack *s,selemtype *d)
{
    //判断栈为空
    if(stack_is_empty(s))
        return false;
    //能获取
    *d = s->top->data;
    return true;

}

/*
```

```
    获取栈中元素的个数
*/
int get_stack_length(linkstack *s)
{
    if(s==NULL)
        return 0;
    return s->num;
}
```

2.队列(queue)

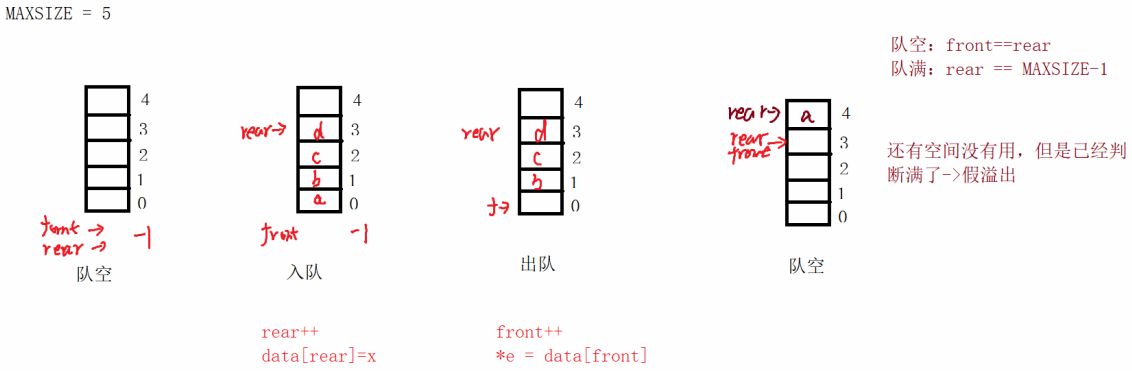
队列是一个先进先出表

只允许在表的一端进行插入，另外一端进行删除

```
队头(front):允许删除的一端
队尾(rear):允许插入的一端
```

2.1顺序队列

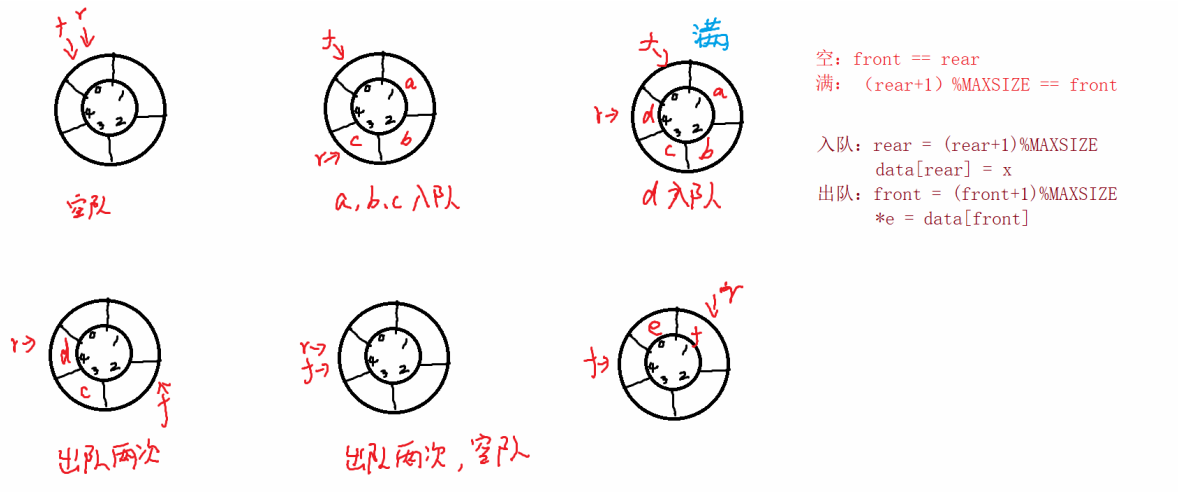
```
#define MAXSIZE 100
typedef int qelemtype;
typedef struct seqqueue
{
    qelemtype data[MAXSIZE];//保存数据的数据
    int front;//队头指针
    int rear;//队尾指针
}seqqueue;
```



2.2循环队列(环形队列)

```
队头指针
队尾指针

队头元素
队尾元素

模型
1. <------上课用的模型
    队头指针：保存的是第一个数据的前一个位置，一直没有存储数据
    队尾指针：保存的是最后一个数据，一直会存放数据
```

```
    队列的长度：(r-f+MAXSIZE)%MAXSIZE
2.
    队头指针：保存的是第一个数据的位置，一直会存放数据
    队尾指针：保存的是最后一个数据的后一个位置，一直没有存储数据
    队列的长度：(r-f+MAXSIZE)%MAXSIZE
多了一个空间判断队空还是满
3.
    队头指针：保存的是第一个数据的位置，一直会存放数据
    队尾指针：保存的是最后一个数据，一直会存放数据
    队列的长度：(r-f+1+MAXSIZE)%MAXSIZE
不需要多利用数据的一个空间判断空或者满
```



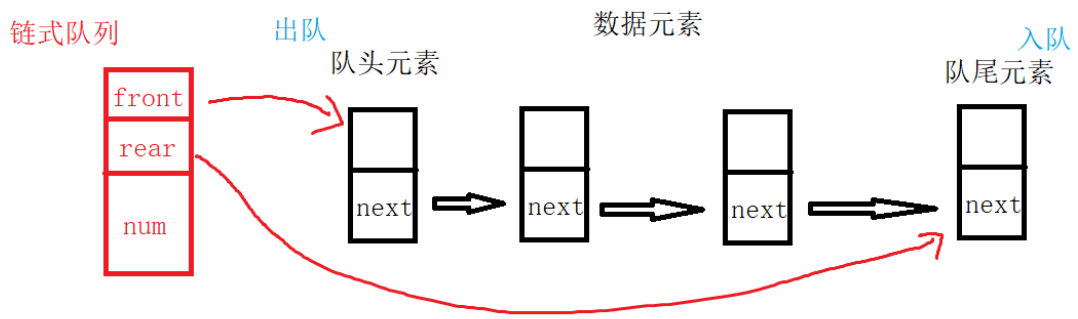
circle_queue.c/circle_queue.h/main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include "circle_queue.h"

/*
    初始化操作，建立一个空队s
    创建一个空队，由s指向它
    实际上就是分配一个顺序队的空间，并将队顶指针设置为-1
*/
circle_queue *init_queue()
{
    circle_queue *q = (circle_queue *)malloc(sizeof(circle_queue));//分配一个顺序队的空间，首地址存放在s
    q->front = q->rear = 0;
    return q;//返回q中存储的地址
}

/*
    若队为空,返回true,否则返回false
*/
bool queue_is_empty(circle_queue *q)
{
    //队不存在 || 一个数据元素都没有
    if(q == NULL || q->front == q->rear)
        return true;
    return false;
}

/*
```

```
    将队清空
    队中没有数据就相当于清空
*/
void clean_queue(circle_queue *q)
{
    //队如果不存在 直接返回
    if(q == NULL)
        return ;

    //队存在，把队变回初始化的样子即可
    q->front = q->rear = 0;//没有数据
}
/*
    若队存在，则销毁它
    释放队s占用的存储空间
*/
circle_queue *destroy_queue(circle_queue *q)
{
    //把申请的空间free即可
    if(q!=NULL)
    {
        free(q);
        q = NULL;
    }

    return q;
}

/*
    入队
    返回值
        true 可以入队
        false 不可以入队
*/
bool enqueue(circle_queue *q,qelemtype e)
{
    //队不存在 或 队满 不能入队
    if(q==NULL || (q->rear+1)%MAXSIZE == q->front)
        return false;

    //入队操作
    q->rear = (q->rear+1)%MAXSIZE;//先让rear+1
    q->data[q->rear] = e;//再把数据放到rear指向的位置
    return true;
}
/*
    出队
    返回值
        true 可以出队
        false 不可以出队

    调用
        qelemtype e;
        pop(s,&e)
*/
bool dequeue(circle_queue *q,qelemtype *d)//d = &e  ==> *d = *&e = e
{
    //调用判断队是否为空的函数,若为true,不能出队
```

```
    if(queue_is_empty(q) == true)
        return false;
    //出队
    q->front = (q->front+1)%MAXSIZE;//front+1
    *d = q->data[q->front];

    return true;
}

/*
    获取队顶元素的值
    返回值
        true 可以获取队顶
        false 不可以获取队顶

    调用
        qelemtype e;
        get_queue_top(s,&e)
*/
int get_queue_front(circle_queue *q,qelemtype *d)
{
    //调用判断队是否为空的函数,若为true,不能出队
    if(queue_is_empty(q) == true)
        return false;

    *d = q->data[(q->front+1)%MAXSIZE];
    return true;
}

/*
    获取队中元素的个数
*/
int get_queue_length(circle_queue *q)
{
    if(q == NULL)
        return 0;
    return (q->rear-q->front+MAXSIZE)%MAXSIZE;
}
```

2.3链式队列




```
//数据类型的定义
typedef int qelemtype;
//队列中的数据结点类型
typedef struct node
```

```
{
    qelemtype data;//数据
    struct node *next;//存放下一个结点的地址
}node;
//链式队列类型
typedef struct linkqueue
{
    node *front;//队头指针
    node *rear;//队尾指针
    int num;//数据结点的个数
}linkqueue;
```

```
#include <stdio.h>
#include <stdlib.h>
#include "linkqueue.h"

/*
    初始化操作，建立一个空队s
    创建一个空队，由s指向它
*/
linkqueue *init_queue()
{
    //为s创建空间
    linkqueue *s = malloc(sizeof(linkqueue));
    //为你的s里面的东西进行赋值
    s->front = s->rear = NULL;
    s->num = 0;
    //返回s
    return s;
}

/*
    若队为空,返回true,否则返回false
*/
bool queue_is_empty(linkqueue *s)
{
    //队不存在 || 队顶为NULL || 数据结点的个数为0
    if(s == NULL || s->front == NULL || s->num == 0)
        return true;

    return false;
}

/*
    将队清空
    队中没有数据就相当于清空
*/
void clean_queue(linkqueue *s)
{
    //判断队是否为空，如果为true ,直接返回
    if(queue_is_empty(s) == true)
        return ;

    //出队(头删)
    while(s->front)
    {
        //1.为你要删除的结点起个名字
```

```
        node *p = s->front;
        //2.更新s->front
        s->front = p->next;
            //发现s->front == NULL,还需要s->rear=NULL
        if(s->front == NULL)
            s->rear = NULL;
        //3.孤立要删除的结点
        p->next = NULL;
        //4.删除
        free(p);
        p = NULL;
        //5.个数--
        s->num--;
    }

}
/*
    若队存在，则销毁它
    释放队s占用的存储空间
*/
linkqueue *destroy_queue(linkqueue *s)
{
    if(s!=NULL)
    {
        //清空队
        clean_queue(s);
        free(s);
        s = NULL;
    }
    return s;
}

/*
    入队
    返回值
        true 可以入队
        false 不可以入队
*/
bool enqueue(linkqueue *s,qelemtype e)
{
    //如果队不存在 返回false
    if(s == NULL)
        return false;

    //队存在  入队(头插)

    //创建结点pnew
    node *pnew = malloc(sizeof(node));
    pnew->data = e;
    pnew->next = NULL;

    //尾插
        //如果队中没有数据结点，新的节点会成为你的front和rear
    if(s->front == NULL)
    {
        s->front = s->rear = pnew;
    }
        //如果有数据结点，做尾插
```

```
    else
    {
        s->rear->next = pnew;
        s->rear = pnew;//更新rear
    }

    //个数+1
    s->num++;
    return true;
}
/*
    出队
    返回值
        true 可以出队
        false 不可以出队

    调用
        qelemtype e;
        pop(s,&e)
*/
bool dequeue(linkqueue *s,qelemtype *d)
{
    //判断队是否为空，如果为true ,直接返回
    if(queue_is_empty(s) == true)
        return false;

    //出队(头删)

    //获取数据
    *d = s->front->data;

    //1.为你要删除的结点起个名字
    node *p = s->front;
    //2.更新s->front
    s->front = s->front->next;
        //发现s->front == NULL,还需要s->rear=NULL
    if(s->front == NULL)
        s->rear = NULL;
    //3.孤立要删除的结点
    p->next = NULL;
    //4.删除
    free(p);
    p = NULL;
    //5.个数--
    s->num--;
    return true;
}

/*
    获取队顶元素的值
    返回值
        true 可以获取队顶
        false 不可以获取队顶

    调用
        qelemtype e;
        get_queue_front(s,&e)
*/
```

```
bool get_queue_front(linkqueue *s,qelemtype *d)
{
    //判断队为空
    if(queue_is_empty(s))
        return false;
    //能获取
    *d = s->front->data;
    return true;

}

/*
    获取队中元素的个数
*/
int get_queue_length(linkqueue *s)
{
    if(s==NULL)
        return 0;
    return s->num;
}
```

3.拓展

3.1 栈的分类

```
满减栈
满增栈
空增栈
空减栈

满：top指针指向的位置，保存了表尾的数据            满栈
空：top指针指向的位置，保存了表尾的数据的下一个     空栈
增：从低地址到高地址依次存储数据
减：从高地址到低地址依次存储数据

代码中用的是满增栈
gcc(arm架构)满减栈
```

3.2 共享栈

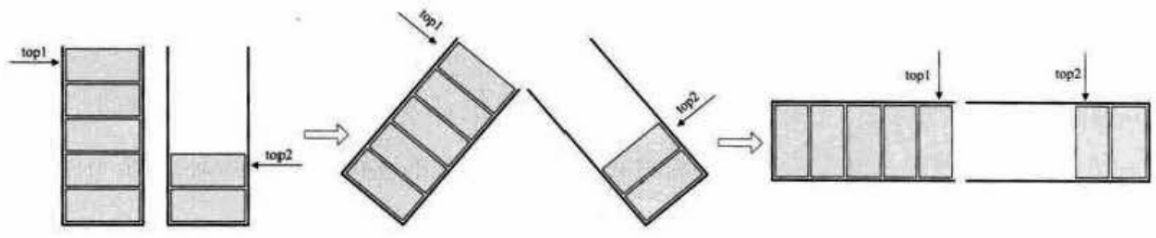


图 4-5-1

```
#define MAXSIZE 100
struct share_stack
{
    int data[MAXSIZE];//放数据
    int top1;//第一个栈的栈顶指针
    int top2;//第二个栈的栈顶指针
};
```

栈空

```
top1 == -1
top2 == MAXSIZE
```

栈满

```
top1 + 1 == top2
```

入栈

```
入栈1
    top1++;
    data[top1] = x
入栈2
    top2--
    data[top2] = x
```

出栈

```
出栈1
    获取top1指向的位置的数据
    top1--
出栈2
    获取top2指向的位置的数据
    top2
```

3.3 中缀表达式求值

(1)中缀转后缀

中缀表达式

```
a+b*c+(d*e+f)*g
```

笔算求后缀表达式

```
1.先把中缀表达式所有的括号打上
    ((a+(b*c))+(((d*e)+f)*g))
2.把当前括号中的运算符放到当前括号的最后面(前缀把运算符放前面)
    ((a(bc*)+)(((de*)f+)g*)+)
3.去掉所有的括号
    abc*+de*f+g*+  (后缀表达式)
```

机算求后缀表达式，规则

```
从左到右遍历中缀表达式中的每个数字和运算符
1.是数字就输出
2.是左括号，直接入栈
3.是右括号，则将栈中的元素依次出栈(打印)，直到遇到左括号为止
    注意：左括号只出栈不打印
4.是其它运算符，跟栈顶元素进行比较优先级
    栈顶元素的优先级高于或等于当前的元素符号，出栈(打印)，出栈到栈顶元素它的优先级低为止，再去把当前的运算符进行入栈
    如果栈顶元素为空，直接入栈
    如果栈顶元素为左括号，直接入栈
5.遍历到中缀表达式的末尾，则将栈中所有的元素依次出栈(打印)
```

(2)后缀表达式求值

```
从左往右遍历后缀表达式
1.数字入栈
2.运算符，出栈两次栈顶元素，计算结果，把结果入栈
    先出栈的为操作数2
    后出栈的为操作数1
eg：
    3+5*8-6/3 =>后缀 3 5 8 * + 6 3 / -
```

(3)中缀表达式求值

机算

```
开辟两个栈，一个运算符栈op，一个数字栈d
出栈一个运算符，出栈两个数字，计算完成把计算的结果入数字栈
```

3.4 栈的应用

```
1.简单的四则运算
2.递归都可以用栈实现
    树的遍历：先序(根)遍历，中序遍历，后序遍历     非递归的算法，需要用栈作为辅助
    图的深度优先搜索
3.括号匹配问题
    (a+b*c)[]
    (a+b*c[]
    一旦遇到左括号( [ { ... 入栈
    遇到右括号，出栈，跟左括号比较，看匹不匹配
```

3.5 队的应用

```
1.树的层次遍历，用队列作为辅助
2.图的广度优先搜索
```